

AUDIOBOOK OS

Enterprise Audiobook Production & Distribution Platform

From System Architecture to a Product Operating Model

PLATFORM BLUEPRINT · v2.0

The Evolution of the Audiobook Production Platform

Prepared by: Mihir Oza

Date: July 23, 2026

Table of Contents

1. Executive Summary

The existing document is a strong system-architecture specification. It defines a clean domain model, provider-independent synthesis, asynchronous workers, storage, security, and phased delivery for an audiobook production platform. That work is preserved in full and remains the technical foundation of everything that follows.

This blueprint takes the next step. It elevates the platform from a **well-designed application architecture** into a complete **product operating model** — a system that a team can build, sell, operate, and evolve into an ecosystem. The reframing is captured in one name: **Audiobook OS**, an operating system for audiobook production.

The central shift

The old mental model ends at a generated file: **React** → **Flask** → **Polly** → **MP3**.

Audiobook OS treats every manuscript, decision, production operation, audio version, review, export, and distribution event as part of a persistent, extensible production record — surrounded by workflows, rules, AI agents, plugins, a marketplace, governance, analytics, and public APIs.

The result is a platform capable of serving publishers, studios, agencies, and audiobook marketplaces, and of adding new speech providers, distribution channels, and AI capabilities without redesigning the product.

1.1 What This Document Adds

The current specification covers architecture, the domain model, APIs, infrastructure, security, and production workflows. A commercial-grade platform needs an additional layer of capabilities. This document supplies each of the following, fully developed:

Missing capability	Delivered as
Product capabilities catalog	Five platform pillars (Section 3)
Workflow engine design	Configurable pipeline engine (Section 4)
Production rules engine	Declarative IF/THEN policy layer (Section 5)
AI orchestration layer	AI Service Mesh of specialized agents (Section 6)
Plugin system	PlatformPlugin contract + lifecycle (Section 7)
Marketplace / extensibility model	Audiobook Marketplace (Section 8)
Provider strategy	Provider Selector with failover (Section 9)
Enterprise controls	Governance, budgets, compliance (Section 10)
Distribution platform	Distribution providers to retailers (Section 11)
Analytics domain	Production + business analytics (Section 12)
Internal automation framework	Cost optimization engine (Section 13)
Observability as product	Visual production timeline (Section 14)
Platform-wide events	Full event-driven architecture (Section 15)

Missing capability	Delivered as
SDK strategy	Public API + multi-language SDKs (Section 16)
Multi-tenant SaaS	Tenant → Org → Project model (Section 17)

1.2 How to Read This Document

Sections 2 and 3 establish the vision and the five pillars that organize every capability. Sections 4 through 17 develop each new subsystem in turn: its purpose, its design, its data model, its interfaces, and how it interacts with the existing architecture. Section 18 provides an integrated delivery roadmap, and Section 19 defines what "done" means for the platform as a product. Wherever this document references an existing element (the domain model, the speech-provider protocol, the job system), it extends rather than replaces it.

2. From Specification to Operating Model

A specification answers the question **"how does the software work?"** An operating model answers a larger question: **"how does the platform evolve into an ecosystem?"** The current document answers the first question well. This section frames the second.

2.1 Gap Analysis

The existing architecture is complete along the dimensions it set out to cover. The table below states plainly what is present and what a production platform additionally requires — the second column is the agenda for the rest of this document.

Already defined (kept in full)	Required to become a platform
Architecture & domain model	Configurable workflow engine
Provider-independent synthesis	Declarative production rules engine
Asynchronous jobs & workers	AI orchestration across specialized agents
Storage, security, observability	Plugin system for third-party extensions
Production workflows (hardcoded)	Marketplace & commercialization model
Single-provider selection	Provider strategy with automatic failover
Export to file formats	Distribution to retailers & stores
Operational logging & metrics	Analytics domain + visual timelines
Internal REST API	Public API, webhooks & multi-language SDKs
Organization/project scoping	True multi-tenant SaaS with billing

2.2 The Reframing: Audiobook OS

The platform should no longer be imagined as a linear utility. Compare the two mental models side by side.

Old model — a utility

```

Upload Book
  |
  v
Generate Audio
  |
  v
Download MP3
  
```

New model — an operating system

```

Content
-> Analysis
-> Production
-> Quality
-> Publishing
-> Distribution
-> Insights
  
```

Each stage of the new model is a durable, observable, and extensible capability rather than a transient step. AWS Polly does not disappear — it becomes one replaceable adapter inside a much larger production system. The API does not disappear — it becomes the command-and-query boundary around a durable domain. The

React frontend does not disappear — it becomes a complete project workspace instead of a set of upload, progress, and download controls.

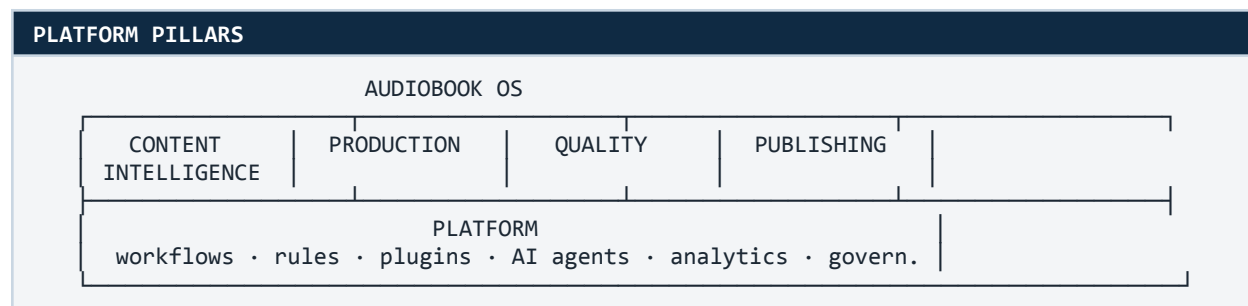
2.3 Design Tenets for the Platform Layer

Every subsystem introduced in this document adheres to the following tenets, which extend the existing architectural principles.

- **Configuration over code.** Publisher-specific behavior (pipelines, rules, provider priority) is data, not a code branch. Different customers configure the platform without a release.
- **Extensibility by contract.** New providers, distributors, and analyzers enter through stable interfaces — the plugin and provider contracts — never by editing core logic.
- **Everything is an event.** Meaningful state changes emit domain events, enabling automation, integrations, webhooks, and analytics without coupling.
- **Governed by default.** Budgets, approvals, compliance, and tenant isolation are enforced by the backend, not merely surfaced in the UI.
- **Traceable end to end.** Any production run can be replayed from user request through provider submission, storage, QC, export, and distribution.

3. The Five Platform Pillars

Audiobook OS organizes every capability — existing and new — into five pillars. The pillars are the product's capability catalog: they give sales, engineering, and operations a shared vocabulary and make the roadmap legible.



3.1 Content Intelligence

Turns raw manuscripts into structured, production-ready scripts.

- Ingestion of text, Markdown, HTML, PDF, DOCX, EPUB, RTF, XML, and scanned PDF via OCR.
- Structural parsing, normalization, and chapter detection with human correction.
- Segmentation into narration, dialogue, quotes, footnotes, poetry, and front/back matter.
- Metadata extraction: title, author, ISBN, language, estimated runtime.

3.2 Production

Converts approved scripts into mastered audio, provider-independently.

- Provider-independent synthesis orchestration with content-addressed reuse.
- Segment, chapter, and full-project synthesis with idempotent retries.
- Audio mastering: silence control, loudness normalization, fades, assembly.
- Non-destructive editing via edit decision lists over versioned audio assets.

3.3 Quality

Guarantees that only correct, compliant audio can be approved.

- Automated audio checks: duration, silence ratio, clipping, loudness, noise floor.
- Text-to-audio verification through transcription comparison.
- Human review workflow with issues, severities, and reviewer assignments.
- Approval gates that block publishing until findings are resolved.

3.4 Publishing

Packages and delivers approved masters to the world.

- Export profiles: MP3, M4B, WAV archive with chapter markers and embedded metadata.
- Distribution to Audible, Apple Books, Spotify, Google Play, Kobo, and more.
- Compliance and legal review: copyright checks, publishing approval, usage audit.
- Export versioning generated only from frozen, approved revisions.

3.5 Platform

The connective tissue that makes the first four pillars configurable and extensible.

- Workflow engine, rules engine, and AI agent mesh.
- Plugin system and marketplace for third-party capabilities.
- Analytics, cost optimization, governance, and multi-tenant SaaS controls.
- Public API, webhooks, and SDKs for programmatic use.

4. Workflow Engine

Today the production lifecycle is hardcoded as a fixed state machine. That works for one process, but publishers, studios, and agencies each run different pipelines. A **workflow engine** makes the production lifecycle a configurable artifact so that each organization — or even each project — defines its own stages, gates, and automation without a code change.

4.1 Why a Workflow Engine

- **Enterprise fit.** Large publishers have mandatory legal and producer reviews; independents want a one-click path.
- **Auditability.** A declared pipeline is a contract: every transition is logged against the workflow definition.
- **Reuse.** Common pipelines become templates shared across an organization's projects.

4.2 The Canonical Pipeline

The default workflow expresses the full lifecycle. Organizations start here and customize.

workflow.yaml

```
workflow: standard_production
stages:
  - ingest
  - chapter_detection
  - ai_review
  - editor_review
  - narrator_assignment
  - synthesis
  - qc
  - approval
  - export
  - distribute
```

4.3 Custom Pipelines by Publisher

The same engine expresses radically different processes. A lightweight independent publisher and a compliance-heavy enterprise publisher use the identical mechanism.

Publisher A — fast path

```
Upload
  |
  v
AI QC
  |
  v
Publish
```

Publisher B — governed path

```
Upload
-> Legal Review
-> Producer Review
-> Narrator Approval
-> Synthesis
-> QC
-> Final Approval
```

4.4 Engine Design

A workflow is a directed graph of stages. Each stage declares its type, the actor or system responsible, entry and exit conditions, and the events it emits. The engine advances a project only when a stage's exit conditions are satisfied, and it records each transition as an audit event.

Concept	Description
Workflow definition	Versioned template of stages, transitions, and gates (per org or project).
Stage	A unit of work: automated (ingest, synthesis, QC) or human (review, approval).
Gate	A condition that must pass to leave a stage (e.g., no blocking QC issues).
Transition	A directed edge; may be automatic, manual, or rule-driven.
Workflow instance	A running project's position and history within a workflow definition.
Hooks	Events emitted on enter/exit for automation, notifications, and analytics.

Relationship to the existing state machine

The current project state machine (draft → ingesting → ... → completed) becomes the **default workflow definition**. The hardcoded transitions are re-expressed as stages and gates so that existing behavior is preserved exactly while new pipelines become possible.

4.5 Data Model

workflow schema

```

WorkflowDefinition
  id
  organization_id
  name
  version
  scope (org | project)
  is_default
  status
  created_at

WorkflowStage
  id
  workflow_definition_id
  key (ingest, qc, ...)
  display_name
  stage_type (auto | human)
  actor_role
  entry_conditions (json)
  exit_conditions (json)
  on_enter_events (json)
  on_exit_events (json)

WorkflowInstance
  id
  project_id
  workflow_definition_id
  current_stage_id
  status
  started_at
  history (json)

WorkflowTransition
  id
  from_stage_id
  to_stage_id
  trigger (auto|manual|rule)
  guard_expression

```

5. Production Rules Engine

Where the workflow engine decides **what happens next**, the **rules engine** decides **how each decision is made**. Today those decisions appear manual. A declarative rules engine encodes organizational policy as data — IF/THEN rules evaluated against project context — so that voice selection, budget approvals, provider routing, and QC severities become consistent, testable, and enforceable.

5.1 Rule Anatomy

A rule has a condition, an action, a scope, and a priority. Rules are grouped into rule sets bound to an organization, project, or workflow stage. The engine evaluates matching rules in priority order and records which rules fired for auditability.

5.2 Representative Rules

Voice rule

```
IF speaker = narrator
THEN voice = matthew_neural
```

Budget rule

```
IF project_cost > $500
THEN require_approval
```

Language rule

```
IF language = spanish
THEN provider = azure
```

QC rule

```
IF silence > 2s
THEN create_critical_issue
```

5.3 Rule Categories

Category	Governs	Example outcome
Voice & casting	Speaker-to-voice mapping	Assign narrator voice by role or character
Budget & approval	Spend thresholds	Require producer sign-off above a limit
Provider routing	Which engine synthesizes	Route a language to a specific provider
Quality control	Issue creation & severity	Escalate excessive silence to critical
Compliance	Legal & policy gates	Force copyright check for licensed titles
Notification	Who is alerted	Notify reviewer when a chapter is ready

Design rule

Rules produce **decisions and recommendations**, never silent irreversible actions. Every fired rule is logged with its inputs and outputs, and rules that spend money or change state route through the workflow's approval gates.

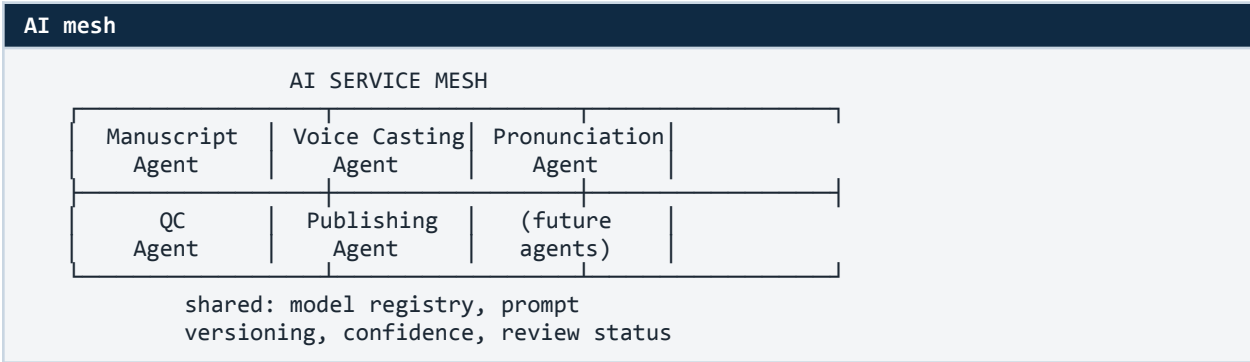
5.4 Data Model

rules schema			
RuleSet		Rule	
id		id	
organization_id		rule_set_id	
project_id (nullable)		name	
stage_key (nullable)		priority	
status		condition_expression (json / DSL)	
created_at		action_type	
		action_payload (json)	
		enabled	
		created_at	
RuleEvaluation			
id			
rule_id	project_id	context_snapshot (json)	
fired (bool)	evaluated_at	result (json)	

6. AI Orchestration Layer

The current design references AI only as a single analysis step. A serious platform treats AI as a coordinated set of specialized services — an **AI Service Mesh** — where each agent owns a narrow responsibility, exposes a uniform contract, and returns recommendations that humans and rules can review. This keeps AI powerful but governed, versioned, and auditable.

6.1 The AI Service Mesh



6.2 Agents and Responsibilities

Agent	Responsibilities
Manuscript Agent	Structure detection, chapter detection, metadata extraction.
Voice Casting Agent	Narrator selection, character-to-voice assignment recommendations.
Pronunciation Agent	Names, locations, and acronyms; suggests dictionary entries.
QC Agent	Transcript comparison, pronunciation validation, mismatch detection.
Publishing Agent	Metadata generation and marketplace-compliance checks.

6.3 Governance of AI Output

Consistent with the existing architectural rule, AI output is a **recommendation, not truth**. Every AI transformation is stored with full provenance so it can be reviewed, reproduced, and rolled back.

A
I
p
r
o
v
e
n
a
n
c
e

A
I
n
f
e
r
e
n
c
e

i
d

a
g
e
n
t
(
m
a
n
u
s
c
r
i
p
t
|
c
a
s
t
i
n
g
|
p
r
o
n
u
n
c
i
a
t
i
o
n
|
q
c
|
p
u
b

l
i
s
h
i
n
g
)

m
o
d
e
l

m
o
d
e
l
-
v
e
r
s
i
o
n

p
r
o
m
p
t
-
v
e
r
s
i
o
n

i
n
p
u
t
-
h
a
s
h

o
u
t
p
u

t
(
j
s
o
n
)

c
o
n
f
i
d
e
n
c
e

r
e
v
i
e
w
-
s
t
a
t
u
s
(
p
e
n
d
i
n
g
|
a
c
c
e
p
t
e
d
|
r
e
j
e
c
t
e
d
|
e

d
i
t
e
d
)

p
r
o
j
e
c
t
-
i
d

c
r
e
a
t
e
d
-
a
t

Non-negotiable

AI analysis must never overwrite source content. Production text remains editable independently of the original manuscript, and every automated transformation carries a confidence score and a review status that gates its use downstream.

6.4 Orchestration Patterns

- **Pipeline invocation.** Workflow stages call agents (e.g., an `ai\_review` stage invokes the Manuscript and Pronunciation agents).
- **Fan-out / fan-in.** Long manuscripts are chunked, analyzed in parallel, and merged with conflict resolution.
- **Provider-agnostic models.** Agents call models through an adapter, so the mesh can mix hosted and internal models the same way synthesis mixes speech providers.
- **Human-in-the-loop.** Low-confidence outputs route to a reviewer before they can advance the workflow.

7. Plugin Architecture

The single largest gap between an application and a platform is extensibility. A **plugin architecture** lets speech providers, distributors, analytics tools, and AI models be added, enabled, and disabled without modifying core code. Plugins are the mechanism that turns Audiobook OS into an ecosystem and, later, a marketplace.

7.1 The Plugin Contract

Every plugin implements a common lifecycle contract. The platform manages installation, enablement, configuration, and teardown uniformly, regardless of what the plugin does.

plugin_contract.py

```
class PlatformPlugin(Protocol):
    name: str
    version: str
    capabilities: list[str]

    async def install(self, context: PluginContext) -> None: ...
    async def enable(self, context: PluginContext) -> None: ...
    async def disable(self, context: PluginContext) -> None: ...
    async def uninstall(self, context: PluginContext) -> None: ...
    async def health(self) -> PluginHealth: ...
```

7.2 Plugin Categories and Examples

Category	Example plugins	Extends
Speech providers	Polly, Azure, ElevenLabs, OpenAI	SpeechProvider protocol
Distribution	Audible Export, Spotify Distribution, YouTube Books	DistributionProvider
Analytics	Usage dashboards, revenue reporting	Analytics domain
AI models	Casting model, QC transcription model	AI Service Mesh
Publishing	Metadata compliance, cover generation	Publishing Agent

Because providers already implement a common `SpeechProvider` interface in the existing architecture, wrapping each provider as a plugin is a natural extension: the plugin registers the adapter and its capability descriptor, and the platform gains a new voice engine without a redeploy.

7.3 Lifecycle and Isolation

plugin lifecycle

```
registered  -> installed  -> enabled  -> disabled  -> uninstalled
  |           |           |           |
  catalog    migrations  active in
  entry      & config    selection/routing
```

- **Sandboxed configuration.** Each plugin declares a typed config schema; secrets are stored per tenant in the secrets manager, never in code.

- **Capability declaration.** Plugins advertise capabilities (e.g., ``ssml``, ``word_timestamps``, ``streaming``) so the provider selector and workflow engine can reason about eligibility.
- **Health & versioning.** Plugins expose health checks and semantic versions; incompatible versions are quarantined rather than loaded.
- **Tenant scoping.** A plugin enabled for one tenant is invisible to others, preserving isolation.

8. Marketplace Architecture

Once plugins exist, a **marketplace** turns extensibility into a distribution and monetization channel. Organizations browse, install, and pay for capabilities the way they do on Salesforce AppExchange, Shopify Apps, or the Atlassian Marketplace — and third parties can publish their own integrations.

8.1 What Can Be Installed

- **Voice providers** — new speech engines and premium voices.
- **QC providers** — specialized quality-control and transcription tools.
- **Analytics providers** — dashboards, benchmarking, and revenue insights.
- **AI models** — casting, pronunciation, and publishing intelligence.
- **Publishing adapters** — distribution to additional retailers and stores.

8.2 Marketplace Model

Element	Description
Listing	Catalog entry: name, category, capabilities, pricing, publisher, reviews.
Publisher account	Verified third party who submits and maintains plugins.
Certification	Contract-test suite each plugin must pass before listing (see §7).
Installation	Tenant-scoped enablement with typed configuration and secrets.
Billing integration	Usage- or subscription-based charges routed through platform billing.
Ratings & governance	Reviews, version history, and revocation for policy violations.

Why it matters commercially

The marketplace converts every new provider or tool from an engineering cost into a revenue-sharing opportunity, and it lets the platform expand faster than any single team could build — the defining characteristic of an ecosystem.

8.3 Certification Pipeline

Every submitted plugin must clear the same behavior contract used for internal adapters before it can be listed.

certification

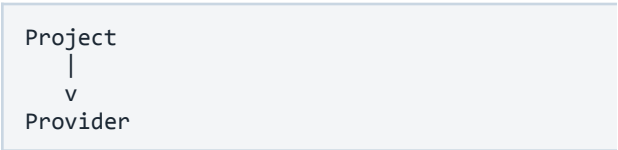
```
submit -> automated contract tests -> security scan ->
manual review -> sandbox trial -> publish -> monitor
```

9. Provider Strategy Layer

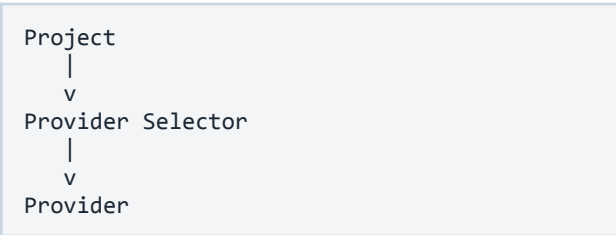
The current design binds a project directly to a provider. Introducing a **Provider Selector** between the project and the provider lets the platform choose the best engine per segment based on cost, quality, latency, language, availability, capabilities, and budget — and fail over automatically when a provider is unavailable.

9.1 Before and After

Current — direct binding



Better — mediated selection



9.2 Selection Signals

The selector scores eligible providers against the segment's requirements and the organization's policy. Inputs include:

- Cost
- Latency
- Availability
- Quality
- Language
- Capabilities
- Budget

9.3 Priority and Automatic Failover

An organization declares an ordered provider priority. If the preferred provider fails or is over budget, the selector automatically falls back to the next eligible provider — a resilience guarantee the current single-binding design cannot offer.

provider priority

y · y
a
m
l

p
r
i
o
r
i
t
y
:

-
e
l
e
v
e
n
l
a
b
s

p
r
e
f
e
r
r
e
d
f
o
r
q
u
a
l
i
t
y

-
a
z
u
r
e

f
a
l
l

b
a
c
k

-
p
o
l
l
y

c
o
s
t
-
e
f
f
e
c
t
i
v
e
b
a
s
e
l
i
n
e

I
f
E
l
e
v
e
n
L
a
b
s
f
a
i
l
s
o
r
i
s
u
n
a

v
a
i
l
a
b
l
e
:

-
>
A
u
t
o
m
a
t
i
c
f
a
i
l
o
v
e
r
t
o
A
z
u
r
e
,
t
h
e
n
P
o
l
l
y

Interaction with the rules engine

Provider-routing rules (§5) constrain the candidate set before scoring — for example, forcing a language to a specific provider — while the selector optimizes within the allowed set and honors the declared failover order.

9.4 Hybrid Routing

Selection can differ per segment within one book, enabling cost/quality optimization impossible under a single binding (developed further in §13):

hybrid routing

Narrator segments	-> ElevenLabs	(premium quality)
Character voices	-> Polly	(cost-effective)
QC transcription	-> OpenAI	(accuracy)

10. Enterprise Governance Layer

Large publishers will not adopt a platform they cannot control. The **governance layer** adds budgets, compliance, and legal review on top of the existing cost-control and audit foundations, so that spend, data handling, and publishing decisions are bounded by policy rather than left to individual discretion.

10.1 Budgets

Budgets are enforced at multiple levels and evaluated before any paid operation. When a budget would be exceeded, the workflow routes to an approval gate rather than proceeding silently.

Budget scope	Enforces
Organization budget	Total spend ceiling across all projects.
Project budget	Per-title spend limit set at project creation.
Monthly budget	Rolling monthly allowance with alerts at defined percentages.
Provider budget	Per-provider caps to control exposure to any one vendor.

10.2 Compliance

The platform is designed to operate within recognized compliance regimes and to produce the evidence auditors require.

GDPR	SOC 2	HIPAA (optional)	ISO 27001
------	-------	------------------	-----------

10.3 Legal Review

Publishing carries legal risk. The governance layer introduces explicit legal checkpoints that can be inserted as workflow stages and enforced by rules.

- **Copyright checks** — verify rights to the manuscript and to any licensed voices before synthesis.
- **Publishing approval** — a mandatory sign-off before distribution to any retailer.
- **Usage audit** — a complete, queryable record of who did what, with what content, at what cost.

Built on existing foundations

Governance does not replace the existing audit log and cost-control architecture — it consumes them. Budgets read from usage records; legal review writes to the audit trail; compliance reporting is generated from the same structured events used for observability.

11. Distribution Platform

The current architecture ends at export — it produces files. Real publishers need to **distribute** those files to the stores where listeners buy them. A distribution subsystem adds a delivery stage after export and models each retailer as a pluggable distribution provider.

11.1 Export Is Not the End

publishing flow

```

Export
|
v
Distribute -> Audible · Apple Books · Spotify Audiobooks
              Google Play Books · Kobo · Libro.fm
              Storytel · Chirp

```

11.2 The Distribution Provider Contract

Mirroring the speech-provider abstraction, every retailer integration implements one interface, so new channels are added as plugins without touching core logic.

distribution_contract.py

```

class DistributionProvider(Protocol):
    name: str
    async def publish(self, export: Export, metadata: Metadata) -> Distribution: ...
    async def unpublish(self, distribution_id: str) -> None: ...
    async def update(self, distribution_id: str, changes: dict) -> Distribution: ...
    async def status(self, distribution_id: str) -> DistributionStatus: ...

```

11.3 Distribution Domain Model

distribution schema

Distribution	DistributionTarget
id	id
project_id	distribution_id
export_id	provider_key (audible, apple, ...)
status	external_id
requested_by	state (pending live removed error)
created_at	submitted_at
completed_at	last_synced_at
	store_url
	error_message

- **Frozen sources only.** Distribution consumes an approved, frozen export version — never a live draft — preserving reproducibility.
- **Per-retailer requirements.** Each provider declares format, loudness, and metadata requirements validated before submission.
- **Lifecycle sync.** Status polling and webhooks keep the platform's view of each store listing current, feeding the analytics domain.

12. Analytics Domain

Analytics are absent from the current design. A platform that publishers pay for must explain **what it produced, what it cost, and how the business is performing**. The analytics domain aggregates the platform's own domain events (see §15) into two complementary views: production analytics for operators and business analytics for decision-makers.

12.1 Production Analytics

Operational metrics that describe how efficiently books move through the pipeline.

Metric	Answers
Characters generated	How much synthesis volume was produced?
Cost by provider	Where is spend concentrated?
Time per book	How long does a title take end to end?
Retry rates	How reliable is each provider and pipeline?
Approval duration	Where do books wait in review?

12.2 Business Analytics

Executive metrics that describe the health and growth of the business.

Metric	Answers
Projects created	Is adoption growing?
Exports generated	How much finished output ships?
Storage growth	How is infrastructure cost trending?
Revenue	What is the platform earning?
Customer usage	Which tenants are most active?

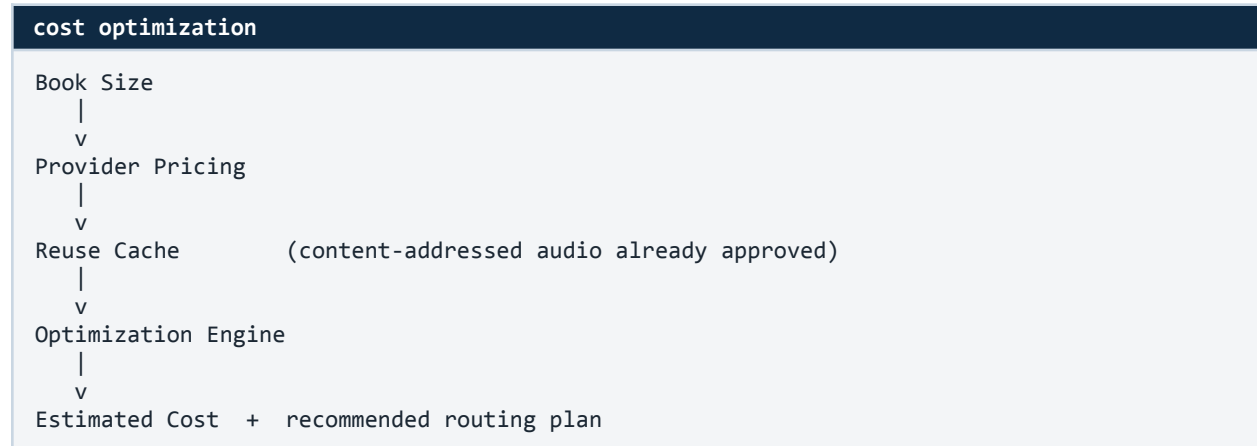
How analytics stays accurate

Analytics are computed from the same immutable domain events that drive automation and audit — not from ad-hoc queries against mutable state. This guarantees that operational dashboards, billing, and executive reports all reconcile to a single source of truth.

13. Cost Optimization Engine

Speech synthesis creates substantial variable cost, and at scale that cost must be actively managed rather than merely reported. The **cost optimization engine** extends the existing cost-control architecture and the provider selector (\$9) with an internal automation framework that estimates, minimizes, and routes spend intelligently before a production run begins.

13.1 The Optimization Flow



13.2 Optimization Modes

Mode	Objective	Typical use
Cheapest provider	Minimize spend	High-volume backlist titles
Fastest provider	Minimize turnaround	Time-sensitive releases
Highest quality	Maximize fidelity	Flagship or premium titles
Hybrid routing	Optimize per segment	Mixed narration and characters

13.3 Hybrid Routing in Practice

The engine can split a single book across providers to balance cost and quality — a direct payoff of the provider-strategy layer.

hybrid plan			
Narrator	->	ElevenLabs	(premium)
Characters	->	Polly	(cost-effective)
QC	->	OpenAI	(accuracy)

- **Reuse first.** The engine credits content-addressed segments already synthesized and approved, so unchanged text is never paid for twice.
- **Estimate before commit.** Before a major run, the platform shows estimated characters, duration, provider charge, segments to generate versus reused, and an estimated completion range.
- **Policy-bounded.** Optimization always respects governance budgets and rules-engine routing constraints.

14. Observability Platform

The existing observability is operational — logs, metrics, and traces for engineers. A platform should also make production **legible to its users**. This section extends observability into a product feature: a visual, replayable production timeline that lets a producer see exactly how a book was made.

14.1 The Production Timeline

Every project exposes a chronological, expandable record of its own production history, assembled from domain events.

production timeline	
Project	
— Manuscript v1	(uploaded, parsed)
— Analysis	(AI review, chapters detected)
— Voice Assignment	(narrator + characters cast)
— Synthesis Run	(segments generated, reused, retrieved)
— QC	(issues opened / resolved)
— Export	(approved master, distributed)

14.2 Capabilities

- **Replay.** Users can visually replay production history to understand how the final master was reached.
- **Drill-down.** Each node expands to the underlying jobs, provider submissions, costs, and QC findings.
- **Correlation.** Timeline entries link to the operational traces (correlation IDs) already produced by the existing observability layer.
- **Shareable.** Reviewers and external stakeholders can be granted read access to a project's timeline for transparency.

Two audiences, one data source

Engineers keep their structured logs, metrics, and distributed traces. The production timeline is the same information, curated and visualized for producers and reviewers. Both are projections of the platform's domain events.

15. Full Event-Driven Architecture

The current design is event-based for jobs. The platform layer extends events **platform-wide**, so that every meaningful domain change is published as an event. Events are the substrate on which automation, integrations, webhooks, and analytics are all built — and they are what keep the many subsystems in this document decoupled.

15.1 Domain Events

A representative — not exhaustive — catalog of domain events:

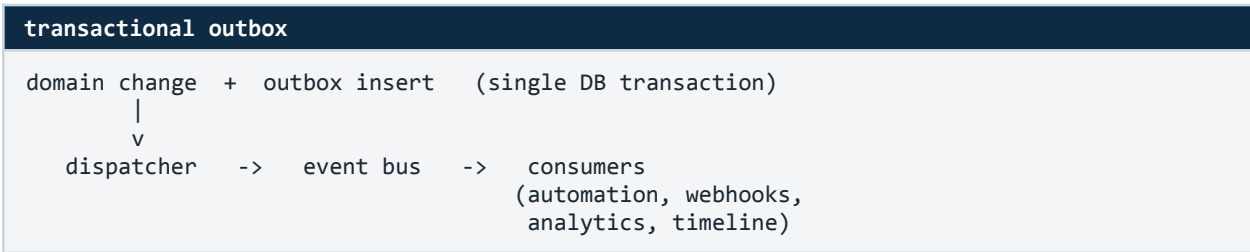
- ProjectCreated
- ManuscriptUploaded
- ChapterDetected
- VoiceAssigned
- SegmentGenerated
- QCIssueOpened
- ChapterApproved
- ExportCompleted
- Published

15.2 What Events Enable

Consumer	Uses events to
Automation	Trigger the next workflow stage or fire a rule.
Integrations	Notify external systems (e.g., a publisher's ERP).
Webhooks	Push real-time updates to customer endpoints.
Analytics	Build production and business metrics (see §12).
Timeline	Assemble the visual production history (see §14).

15.3 Reliability

To avoid losing events between a database commit and the message bus, every domain change writes its event to a **transactional outbox** in the same transaction; a dispatcher then publishes it. This pattern — already recommended in the existing architecture — becomes the backbone of the platform-wide event system, guaranteeing at-least-once delivery with idempotent consumers.



16. Public API and SDK Strategy

The current architecture defines internal APIs. To become a platform, Audiobook OS must be programmable by customers and partners. This section adds a **public API** surface and first-party **SDKs**, turning the product into something other software can build upon.

16.1 Public API Surface

Surface	Purpose
REST	Resource-oriented access to projects, manuscripts, production, and exports.
GraphQL	Flexible, client-driven queries for rich dashboards and integrations.
Webhooks	Event-driven push of domain events to customer endpoints (see §15).

The public API is versioned, authenticated per tenant, rate-limited, and generated from the same contract as the internal API so that frontend and SDK types stay in sync.

16.2 First-Party SDKs

TypeScript	Python	.NET	Go
------------	--------	------	----

An SDK should make common flows a few lines of code:

```
python sdk
```

```
project = client.projects.create(
    title="The Silent Sea",
    language="en-US",
)
project.upload_manuscript("manuscript.docx")
project.start_production()
```

- **Generated, not hand-written.** SDK models are generated from the API contract, eliminating drift.
- **Event-aware.** SDKs expose webhook verification and typed event payloads.
- **Marketplace synergy.** The same SDKs let third parties build the plugins that populate the marketplace (§8).

17. Multi-Tenant SaaS Architecture

If Audiobook OS is commercial, it must serve many customers from one platform with strict isolation and per-customer billing. The existing model scopes data to organizations and projects; multi-tenancy adds a **tenant** above them and the operational machinery of a SaaS business.

17.1 Tenancy Hierarchy



A tenant is the billing and isolation boundary. An existing organization becomes a unit inside a tenant, so the current authorization model — which begins every check with organization membership — extends cleanly by adding a tenant scope above it.

17.2 SaaS Capabilities

Capability	Description
Quota enforcement	Per-tenant limits on projects, storage, synthesis volume, and API calls.
Billing	Subscription and usage-based charges, invoices, and marketplace revenue share.
Tenant isolation	Logical separation of data, secrets, and storage prefixes per tenant.
Tenant analytics	Per-tenant dashboards feeding the analytics domain (see §12).
Custom branding	Tenant-specific logos, domains, and themes in the workspace.

Isolation is non-negotiable

Consistent with the existing security architecture, no job or request may ever access another tenant's records. Tenant scoping is enforced at the data layer, storage-prefix layer, and secrets layer — never merely hidden in the UI — and cross-tenant access attempts are alerted on.

17.3 How the Pieces Fit

Multi-tenancy is where every subsystem in this document converges: workflows and rules are configured per tenant; plugins and marketplace installs are tenant-scoped; provider strategy, budgets, and analytics are evaluated per tenant; and the public API authenticates and rate-limits per tenant. The tenant is the seam that lets one codebase serve a publisher, a studio, and an agency simultaneously — each with its own process, providers, and economics.

18. Integrated Delivery Roadmap

The platform capabilities in this document build on top of the existing ten-phase engineering plan; they should not begin before that foundation exists. The roadmap below sequences the **platform layer** into four waves, each delivering standalone value while setting up the next.

Wave	Focus	Delivers	Depends on
Wave 1	Configurable core	Workflow engine, rules engine, platform-wide events	Existing Phases 1–3
Wave 2	Intelligence & routing	AI Service Mesh, provider selector, cost optimization	Wave 1
Wave 3	Reach & insight	Distribution platform, analytics, production timeline	Existing Phases 6–8, Wave 2
Wave 4	Ecosystem & scale	Plugins, marketplace, public API + SDKs, multi-tenant SaaS	Waves 1–3

18.1 Wave 1 — Configurable Core

- Re-express the hardcoded state machine as the default workflow definition; add custom workflows.
- Introduce the rules engine and migrate manual decisions into declarative rules.
- Extend the transactional outbox into a platform-wide domain-event bus.

18.2 Wave 2 — Intelligence and Routing

- Stand up the AI Service Mesh with the Manuscript, Casting, Pronunciation, QC, and Publishing agents.
- Insert the provider selector between projects and providers; add priority and failover.
- Ship the cost optimization engine with estimate-before-commit and hybrid routing.

18.3 Wave 3 — Reach and Insight

- Add distribution providers for the major retailers behind the DistributionProvider contract.
- Build the analytics domain (production + business) from domain events.
- Deliver the visual production timeline as a first-class workspace feature.

18.4 Wave 4 — Ecosystem and Scale

- Ship the plugin system and wrap existing providers as plugins.
- Launch the marketplace with a certification pipeline.
- Publish the public API, webhooks, and first-party SDKs.
- Introduce the tenant boundary, quotas, billing, isolation, and branding.

19. Definition of Platform Completion

The existing document defines when the **architecture** is production-grade. The platform can be considered complete when, in addition, the system can demonstrate all of the following:

1. A publisher can define and run its own production workflow without a code change.
2. Organizational policy — voice, budget, routing, QC — is enforced by declarative rules.
3. AI agents produce reviewable, versioned recommendations that never overwrite source content.
4. A new speech provider or retailer can be added as a plugin without redeploying the core.
5. Third parties can publish certified plugins to a marketplace and be billed for them.
6. Synthesis is routed across providers by cost, quality, latency, and availability with automatic failover.
7. Budgets, compliance regimes, and legal approvals bound every paid and published action.
8. Approved masters are distributed to major retailers and their status is tracked.
9. Production and business analytics reconcile to a single event-sourced source of truth.
10. Any project's production history can be visually replayed end to end.
11. Customers can operate the platform programmatically via public API, webhooks, and SDKs.
12. Many tenants are served from one platform with strict isolation, quotas, and billing.

The one-sentence test

The platform is complete when Audiobook OS can add a new speech provider, a new distribution channel, and a new AI capability — for a new customer with a different production process — without redesigning the product or shipping bespoke code.

20. Closing: The Final Transformation

The original architecture correctly rejected the naive model:

not this

```
React -> Flask -> Polly -> MP3
```

This blueprint completes that trajectory. The platform is a durable domain surrounded by a configurable, extensible operating layer:

AUDIOBOOK OS

```
Tenant (billing & isolation)
-> Publishing organization
-> Audiobook project (workflow-driven, rule-governed)
-> Versioned manuscript -> AI-assisted analysis
-> Voice, speaker & pronunciation configuration
-> Provider-selected, cost-optimized production
-> Versioned audio assets -> mastering
-> Automated + human quality control & approvals
-> Frozen approved exports
-> Multi-retailer distribution
-> Analytics, timelines & insights
— all extensible via plugins, a marketplace, and public APIs —
```

AWS Polly remains useful, but it is now one replaceable adapter among many. The API remains important, but it is the command-and-query boundary around a durable domain and a public contract for partners. The frontend remains React-based, but it is a complete project workspace with editing, review, monitoring, analytics, and timelines.

The central architectural transformation

Stop treating generated audio as the product result. Treat every manuscript, decision, production operation, audio version, review, export, and distribution as part of a persistent, extensible audiobook production record — and wrap that record in the workflows, rules, intelligence, governance, and ecosystem that let it evolve.

That is the architecture that carries the product from a functioning MVP, through a serious publishing platform, to Audiobook OS: an operating system for audiobook production capable of serving publishers, studios, agencies, and marketplaces, and of absorbing every future provider, channel, and AI capability without being redesigned.